

SPECIAL LECTURE

Local LLM (On-Premise)

ระบบโมเดลภาษาขนาดใหญ่ในองค์กร

ร้อยเอก ดร.ชัชรินทร์ เลิศยศบดินทร์

เป้าหมาย

01

เข้าใจความแตกต่างระหว่าง Cloud LLM กับ Local LLM และรู้ว่าเมื่อไรควรเลือกแบบไหน

02

รู้จัก llama.cpp, GGUF, และแนวคิด quantization

03

ติดตั้ง LM Studio สำเร็จ และดาวน์โหลดโมเดลอย่างน้อย 1 ตัวจาก Hugging Face

04

คำนวณ/ประมาณได้ว่าเครื่องของตัวเองเหมาะกับโมเดลขนาดไหน

05

ตั้งค่า Context Length, GPU Offload, CPU Thread ได้อย่างมีประสิทธิภาพ

06

ตั้งค่า Temperature, Top K, Top P, Repeat Penalty ให้เหมาะกับงานที่ต้องการ

เปรียบเทียบ Cloud LLM vs Local LLM

ประเด็น	Cloud LLM	Local LLM
ความเป็นส่วนตัวของข้อมูล	ข้อมูลถูกส่งไปยังเซิร์ฟเวอร์ผู้ให้บริการ	ข้อมูลไม่ออกจากเครื่อง เหมาะกับข้อมูล sensitive
ค่าใช้จ่าย	จ่ายตามการใช้งาน (token)	ลงทุนฮาร์ดแวร์ครั้งเดียว ใช้ฟรีหลังจากนั้น
การใช้งานแบบ offline	ต้องมีอินเทอร์เน็ต	ใช้ได้แม้ไม่มีเน็ต
ความฉลาด / คุณภาพ	โมเดลใหญ่ที่สุด ทันสมัยที่สุด	จำกัดตามสเปกเครื่อง มักใช้โมเดลเล็กกว่า
ความเร็ว	เร็วสม่ำเสมอ (มี GPU ฟาร์มรองรับ)	ขึ้นกับ CPU/GPU ของเครื่องผู้ใช้
การปรับแต่ง	จำกัดตาม API ที่เปิดให้	ปรับ context, sampling, quantization ได้เต็มที่

Local LLM เหมาะกับใคร?

01

ข้อมูลที่มีความอ่อนไหว

เหมาะสำหรับงานด้านเอกสาร
ภายในองค์กร หรือการจัดการ
ข้อมูลส่วนบุคคลของลูกค้า
ที่ไม่สามารถเปิดเผยภายนอกได้

02

การทดลองและพัฒนา

เหมาะสำหรับการสร้างและ
ทดสอบแอปพลิเคชันต้นแบบ
โดยไม่ต้องกังวลเรื่องค่าใช้จ่าย
API ที่อาจบานปลายระหว่างพัฒนา

03

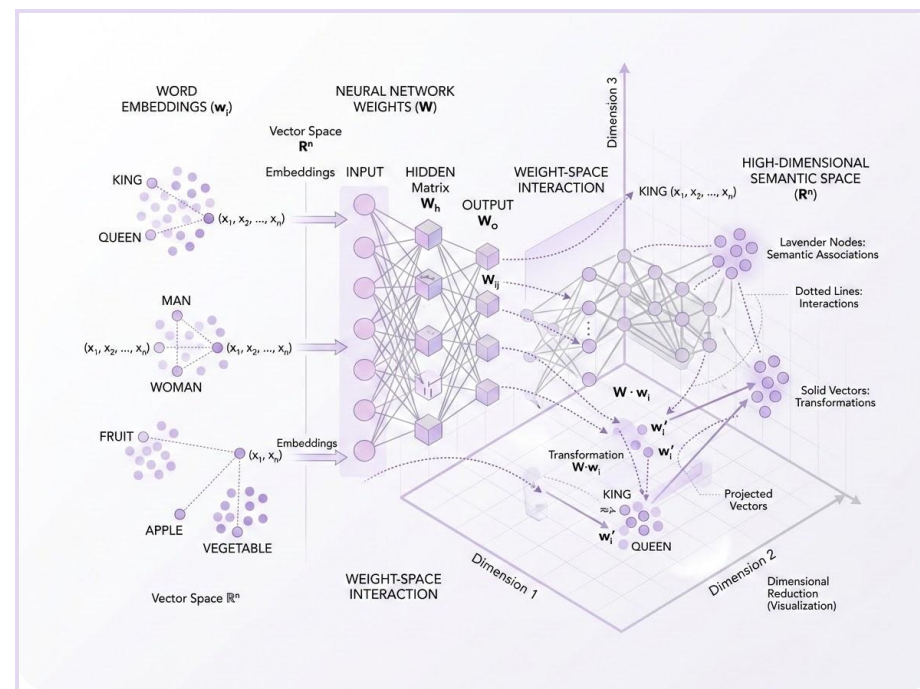
พื้นที่ข้อจำกัดทางเครือข่าย

ตอบโจทย์การใช้งานในห้องปฏิบัติการ
คอมพิวเตอร์ส่วนบุคคล หรือพื้นที่
ที่ไม่มีสัญญาณอินเทอร์เน็ตที่เสถียร

คำศัพท์ที่ต้องทราบ

Inference

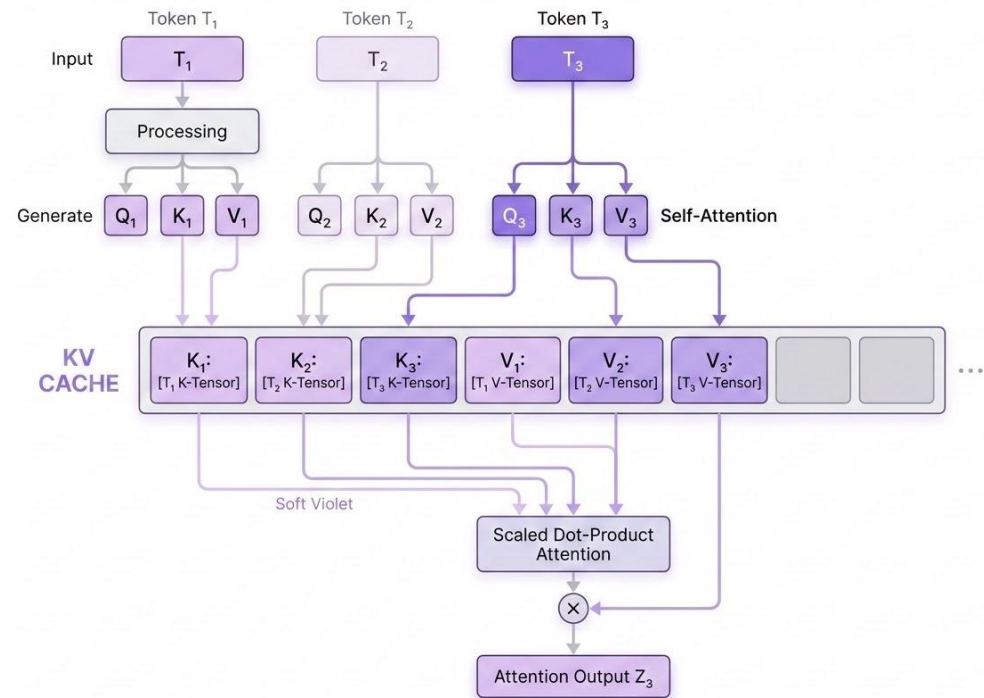
- การ "รันโมเดล"
- การนำ weights มาคูณกับ input (prompt ของเรา) ทีละขั้น
- เพื่อทำนาย token (หน่วยย่อยของคำ) ถัดไปเรื่อยๆ



คำศัพท์ที่ต้องทราบ

KV Cache

- พื้นที่หน่วยความจำที่เก็บค่า Key (K) และ Value (V) ที่คำนวณไว้แล้วของทุก token ก่อนหน้าในบทสนทนา
- เพื่อนำกลับมาใช้ซ้ำในกลไก Self-Attention ของแต่ละเลเยอร์โดยไม่ต้องคำนวณใหม่
- ช่วยลดระยะเวลาในการประมวลผล (Inference) และประหยัดทรัพยากรของ GPU ได้



คำศัพท์ที่ต้องทราบ

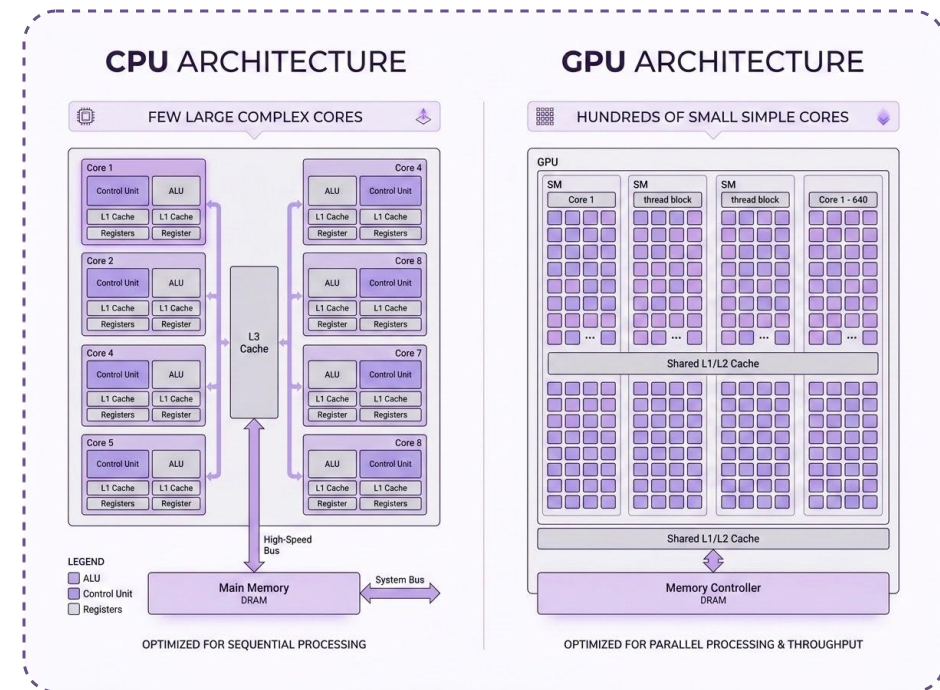
สถาปัตยกรรม CPU และ GPU

CPU (Central Processing Unit)

- มี 4 – 10 Core
- แต่ละ core "ฉลาด" มาก มี control logic ซับซ้อน
- ทำงานทีละงานให้เสร็จเร็วที่สุด เหมาะกับงานคิดเป็นลำดับขั้น ซับซ้อน

GPU (Graphics Processing Unit)

- มีคอร์จำนวนมาก (มากกว่า 10 Core)
- ทำงานพร้อมกันได้หลายงานในเวลาเดียวกัน
- ผ่านโมเดล SIMT (Single Instruction, Multiple Threads)
- สั่งกระจายให้ทุก core ทำงานพร้อมกันกับข้อมูลคนละก้อน



คำศัพท์ที่ต้องทราบ

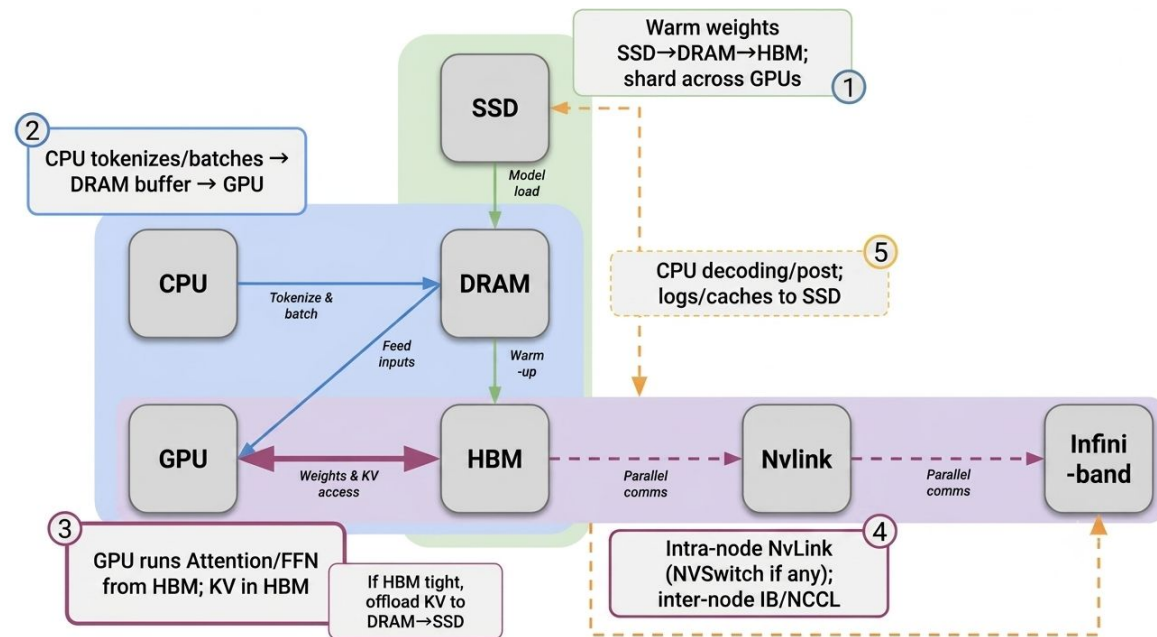
สถาปัตยกรรม RAM และ VRAM

มิติเปรียบเทียบ	RAM (System Memory)	VRAM (GPU Memory)
ชนิดหน่วยความจำทั่วไป	DDR4 / DDR5	GDDR6 / GDDR6X (การ์ดผู้บริโภคร), HBM2e/HBM3 (การ์ดระดับ datacenter)
Bandwidth โดยประมาณ	หลักสิบ GB/s (DDR5 dual-channel ทั่วไป ~60-90 GB/s)	หลักร้อยถึงกว่าพัน GB/s (GDDR6X ~800-1000 GB/s, HBM3 อาจเกิน 3000 GB/s)
ความจุทั่วไป (เครื่องผู้บริโภคร)	8-64 GB (โน้ตบุ๊กทั่วไป)	4-24 GB (การ์ดจอผู้บริโภคร), 80-192 GB (GPU ระดับองค์กร)
บทบาทในงาน LLM inference	เก็บ weights ส่วนที่ยังไม่ offload + เป็นที่พักข้อมูล ก่อนส่งให้ GPU	เก็บ weights ของเลเยอร์ที่ offload ไปประมวลผลจริง + KV cache ส่วนที่ GPU ดูแล

คำศัพท์ที่ต้องทราบ

llama.cpp

- โปรแกรม inference engine ที่เขียนด้วยภาษา C/C++
- ให้รัน LLM ได้บนฮาร์ดแวร์ทั่วไปได้
- เนื่องจากฮาร์ดแวร์แต่ละแบบมีวิธีเรียกใช้
พลังการคำนวณที่ต่างกัน



คำศัพท์ที่ต้องทราบ

CUDA (NVIDIA)

- แพลตฟอร์มคำนวณของ GPU NVIDIA
- แบ่งเป็นหน่วยย่อยเรียกว่า Streaming Multiprocessor (SM)
- แต่ละ SM บรรจุ CUDA core (ทำการคำนวณทั่วไป) และ Tensor core (ทำเฉพาะการคูณเมทริกซ์)
- มี warp scheduler ที่คอยสลับกลุ่มงานเข้าออกไม่ให้ core ว่าง
- ได้รับการซัพพอร์ตดีที่สุดและเร็วที่สุดใน llama.cpp เมื่อเทียบกับ backend อื่นๆ



คำศัพท์ที่ต้องทราบ

Vulkan

- API กลางที่ดูแลโดยองค์กรมาตรฐาน Khronos Group
- ไม่ผูกกับยี่ห้อใดยี่ห้อหนึ่ง
- ใช้ได้ทั้งการ์ด NVIDIA, AMD, Intel
- มักถูกใช้เป็น "ทางเลือกสำรอง" เมื่อไม่มี backend เฉพาะยี่ห้อให้ใช้



ROCm(AMD)

- แพลตฟอร์มคำนวณของ GPU AMD
- ใช้ภาษาโปรแกรม (HIP) ที่ AMD ออกแบบมาให้ใกล้เคียง CUDA



คำศัพท์ที่ต้องทราบ

MLX (Apple Silicon)

- ออกแบบมาสำหรับสถาปัตยกรรม Unified Memory ของชิป Apple Silicon (M-series)
- CPU และ GPU ใช้หน่วยความจำ "ก้อนเดียวกัน"
- ทำให้ Mac ชิป Apple Silicon รันโมเดลขนาดใหญ่ได้อย่างมีประสิทธิภาพ
- เพราะ RAM ทั้งเครื่องทำหน้าที่เหมือน VRAM ไปในตัว

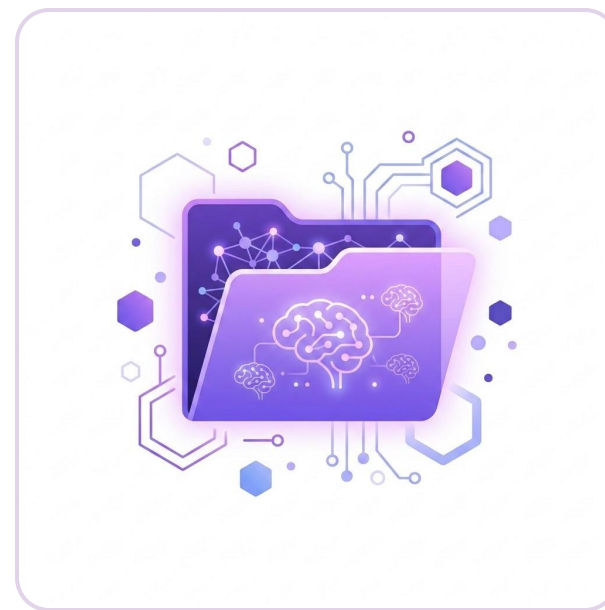


คำศัพท์ที่ต้องทราบ

GGUF

GPT-Generated Unified Format

- พอร์มัตไฟล์ที่ llama.cpp ใช้เก็บโมเดล
- รวมทุกอย่างที่จำเป็นไว้ใน "ไฟล์เดียว"
 - คำ weights
 - ข้อมูล tokenizer (ตัวแบ่งคำ)
 - และ metadata อื่น ๆ (จำนวน layer, จำนวน hidden node)



การเลือก LLM ให้เหมาะสม

หัวใจสำคัญ

การเลือกใช้ Local LLM ให้ได้ผลดี คือการเลือกโมเดลที่ **"พอดี"** กับเครื่องขององค์กร ไม่ใหญ่เกินไปจนรันไม่ไหว และไม่เล็กเกินไปจนคำตอบไม่ได้เรื่อง

1. ชนิดของโมเดล

เลือกโมเดลตามรูปแบบการใช้งานและวัตถุประสงค์ เช่น โมเดลสำหรับสนทนาโต้ตอบทั่วไป (Instruct model) หรือโมเดลเขียนโค้ด (Coder model)

2. ขนาดของโมเดล

พิจารณาจำนวนพารามิเตอร์ (เช่น 1-3B, 8B, 14B) เพื่อให้เหมาะสมกับสเปกคอมพิวเตอร์และหน่วยความจำ (RAM/VRAM) ของเรา

เลือก "ชนิด" ของโมเดล

Base model

โมเดลดิบที่เทรนแค่ "ทำนายคำถัดไป" ยังไม่ถูกสอนให้
สนทนา ไม่เหมาะทำแชทบอทโดยตรง

Instruct model

ถูก fine-tune ให้ทำตามคำสั่งและสนทนาโต้ตอบได้

Coder model

เชี่ยวชาญการเขียน/อธิบายโค้ดโดยเฉพาะ

Reasoning model

เก่งด้านคิดเป็นขั้นตอน (chain-of-thought) เหมาะกับ
โจทย์คณิตศาสตร์/ตรรกะ แต่มักตอบช้ากว่า

เลือก "ขนาด" ของโมเดล

ขนาดโมเดลวัดเป็นจำนวนพารามิเตอร์ (B = พันล้าน) • ยิ่งเยอะยิ่งฉลาดขึ้นแต่กินทรัพยากรมากขึ้นและประมวลผลช้าลง

1–3B

ตัวอย่างโมเดล
Qwen3 1B/3B,
Typhoon 2 1B/3B,
Llama 3.2 3B

ไฟล์ Q4_K_M
0.6–2 GB

เหมาะสำหรับ
ไม้นับพันตัวไป, RAM
8GB, ไม่มี GPU แยก

4–8B

ตัวอย่างโมเดล
Gemma 3 4B,
Qwen3 4B/8B,
Typhoon 2 7B/8B

ไฟล์ Q4_K_M
2.5–5 GB

เหมาะสำหรับ
RAM 16GB หรือ GPU
6–8GB VRAM

14B

ตัวอย่างโมเดล
Phi-4 14B,
Qwen3 14B

ไฟล์ Q4_K_M
~9 GB

เหมาะสำหรับ
RAM 32GB หรือ GPU
10–12GB VRAM

32B+

ตัวอย่างโมเดล
Qwen3 32B

ไฟล์ Q4_K_M
~19 GB

เหมาะสำหรับ
เวิร์กสเตชัน/GPU
24GB VRAM ขึ้นไป

70B

ตัวอย่างโมเดล
Typhoon 2 70B

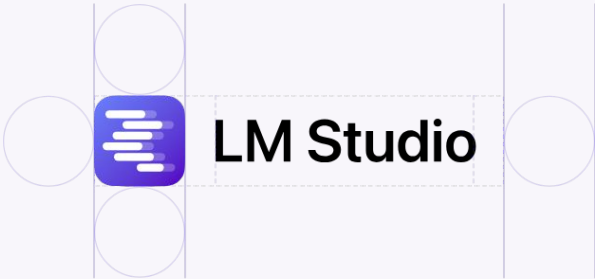
ไฟล์ Q4_K_M
~40 GB

เหมาะสำหรับ
เซิร์ฟเวอร์/GPU
หลายตัว ระดับองค์กร

Quantization – บีบอัดโมเดลให้เล็กลง

F16	~14.0 GB	เต็มความแม่นยำ (baseline)
Q8_0	~7.5 GB	แทบไม่ต่างจากต้นฉบับ
Q6_K	~5.8 GB	ใกล้เคียงมาก
Q5_K_M	~4.9 GB	ลดลงเล็กน้อยแทบสังเกตไม่ได้
Q4_K_M	~4.1 GB	สมดุลที่สุด – นิยมใช้มากที่สุด
Q3_K_M	~3.3 GB	เริ่มเห็นคุณภาพลดลงชัดเจน
Q2_K	~2.8 GB	ลดลงมาก ใช้เมื่อจำเป็นจริง ๆ

เครื่องมือที่ใช้รัน Local LLM อย่างง่าย



- LM Studio เป็นแอปเดสก์ท็อปฟรี
- รองรับ Windows, macOS และ Linux

สเปกเครื่องขั้นต่ำที่แนะนำ

รายการ	ขั้นต่ำ	แนะนำ
RAM	8 GB	16 GB ขึ้นไป
พื้นที่ดิสก์	10 GB ว่าง	50 GB+ (สำหรับเก็บหลายโมเดล)
OS	Windows 10/11 (x64/ARM), macOS 13+, Ubuntu 20.04+	—
GPU	ไม่จำเป็น (รันบน CPU ได้)	NVIDIA/AMD GPU 6GB+ VRAM หรือ Apple Silicon (M-series)

Update available

Models



My Models

Arch

phi2

qwen2

qwen35

qwen3

gemma4

Search local models by name or author...

Staff picks ↻

Most Likes ↕



GPT-OSS 20B

The 20B variant of OpenAI's open source model. ... 351 days ago



Gemma 4 E4B

Gemma4, effective 4B version. Supports image in... 103 days ago



Gemma 4 26B A4B

Gemma 4 is the latest on-device model family fro... 111 days ago



Qwen3.5 9B

Qwen3.5 represents a significant leap forward, int... 142 days ago



Gemma 4 31B

31B dense Gemma4 variant. Supports vision inpu... 103 days ago



Gemma 4 E2B

Gemma4, effective 2B version. Supports image in... 103 days ago



Deepseek R1 0528 Qwen3 8B

Distilled version of the DeepSeek-R1-0528 model... 420 days ago



Qwen3 Coder 30B



openai/gpt-oss-20b



1,604,544

304

Last updated: 327 days ago

Staff Pick ↗

The 20B variant of OpenAI's open source model. Apache 2.0 licensed.

Params

20B

Arch

gpt-oss

Domain

llm

Format

GGUF

MLX

Capabilities:

Tool Use

Reasoning

Download Options

GGUF

GPT-OSS

20B

MXFP4

12.11 GB

Partial GPU Offload Possible

Download 12.11 GB

README

gpt-oss

Designed for lower latency and specialized or local deployment, the model has 21B total parameters with only 3.6B active at a time. Thanks to native MXFP4 quantization for the MoE layer, it runs efficiently and it's capable of operating within 16GB of memory.

This model is released under a permissive Apache 2.0 license and it features configurable reasoning effort—low, medium, or high, so users can balance output quality and latency based on their needs. The model offers full chain-of-thought visibility to support easier debugging and increased trust, though this output is not intended for end users. It is fully fine-tunable, enabling

Override Domain Type ?



google.gemma-4-e4b-it



Chats



Search chats...



New Folder



Unnamed Chat

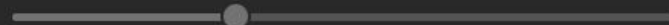
0 tokens



Context Length

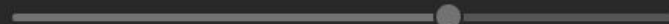
4096

Model supports up to 131072 tokens



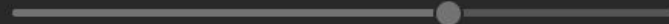
GPU Offload

28



CPU Thread Pool Size

7



Evaluation Batch Size

2048



Physical Batch Size

512



Max Concurrent Predictions

Experimental

4

Reload to apply changes



Model Parameters

Preset

Select a Preset...

+ Save Preset As...



Context Overflow

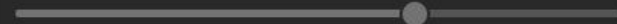
truncate middle

Stop Strings

Enter a string and press ↵

CPU Threads

7



Sampling

Top K Sampling

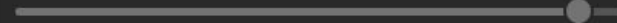
40

Repeat Penalty ☒

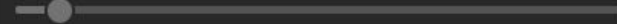
1.1

Presence Penalty ☐Top P Sampling ☒

0.95

Min P Sampling ☒

0.05



Structured Output



Speculative Decoding



Send a message to the model...



Vision



0%

User Ctrl + U

Insert Ctrl + I





Status: Running

Server Settings

mcp.json

Reachable at: `http://192.168.56.1:1234`

+ Load Model

Loaded Models

Server Port

Require Authentication ☐

Active API Keys: 0

Manage Tokens

Serve on Local Network ☒

Allow per-request MCPs ☒

Allow calling servers from mcp.json ☐

Enable CORS ☒

Just-in-Time Model Loading ☒

Auto unload unused JIT loaded models ☒

Max idle TTL minutes

Only Keep Last JIT Loaded Model ☒

apps or from your code.
load a model

Supported endpoints

LM Studio API OpenAI-compatible

- GET /v1/models
- POST /v1/responses
- POST /v1/chat/completions
- POST /v1/completions
- POST /v1/embeddings

NEW REST API v1

▼

No model selected

Developer Logs

More

Clear

Copy

▼

การตั้งค่า Model (Load Settings)



Context Length

- คือจำนวน token สูงสุดที่โมเดลจะ "จำ" ได้ในบทสนทนาเดียว
- ยิ่งตั้งค่า context สูง ยิ่งใช้ KV cache สูงมากขึ้นตามไปด้วย
แม้ตัวไฟล์โมเดลจะมีขนาดเท่าเดิม

การตั้งค่า Model (Load Settings)

GPU Offload

- GPU Offload คือการเลือกที่จะย้ายที่เลเยอร์ไปประมวลผลบน GPU (**เร็วกว่า**)
- ส่วนที่เหลือจะประมวลผลบน CPU
- ถ้าตั้งค่าเป็น 0 หมายถึง รันด้วย CPU ล้วน ๆ
- ยิ่ง Offload ไปที่ GPU ได้มากเท่าไร ก็ยิ่งเร็วขึ้นเท่านั้น

การตั้งค่า Model (Load Settings)



CPU Thread Pool Size

- จำนวน thread ของ CPU ที่ใช้ประมวลผล
- การตั้งค่าจะต้องเหลือ core ไว้ให้ OS ทำงานอื่นด้วย

การตั้งค่า Model (Load Settings)



Evaluation & Physical Batch Size

- **Evaluation Batch Size:** คือจำนวน token สูงสุดที่ประมวลผล prompt พร้อมกันได้ในช่วง ๓SSกะ
- **Physical Batch Size:** คือขนาด batch จริงที่ถูกส่งเข้าฮาร์ดแวร์ต่อรอบ (มักเล็กกว่า evaluation batch)
- การตั้งค่า Physical Batch Size ที่ สูงขึ้น จะช่วยให้การประมวลผล prompt ยาว ๆ ทำงานได้เร็วขึ้นอย่างเห็นได้ชัด

การตั้งค่าการแชท (Sampling Settings)



Temperature

คือตัวคุมว่าจะเลือกแบบมั่นใจ (เลือกคำที่น่าจะเป็นที่สุดเสมอ)
หรือแบบสุ่มเสียง (เปิดโอกาสให้คำที่ไม่ได้น่าจะเป็นที่สุดถูกเลือกได้บ้าง)

- **Temperature ต่ำ (0.1–0.3):** คำตอบสม่ำเสมอ ตรงไปตรงมา เหมาะกับงานแปล สรุป หรือดึงข้อมูล
- **Temperature กลาง (0.7–0.9):** สมดุล เหมาะกับแชทบอกรทั่วไป (ค่าเริ่มต้นมักอยู่ที่ 0.8)
- **Temperature สูง (1.2–2.0):** สร้างสรรค์ หลากหลาย แต่เสียงหลุดประเด็นหรือไม่สมเหตุสมผล เหมาะกับงานแต่งเรื่อง

การตั้งค่าการแชท (Sampling Settings)



Top K & Top P

Top K

คือการจำกัดว่าโมเดลเลือกได้จากคำที่น่าจะเป็นที่สุด K อันดับแรกเท่านั้น
(เช่น K=40 คือเลือกจาก 40 คำที่มีความน่าจะเป็นสูงสุด)

Top P

คือการเลือกกลุ่มคำที่ความน่าจะเป็นรวมกันได้ถึง P% (เช่น 0.95 = 95%)

*ทั้งสองค่าทำงานร่วมกันเพื่อ "ตัดหาง" คำที่ไม่น่าจะเป็นไปได้ทิ้ง ก่อนจะสุ่มเลือกในกลุ่มที่เหลือด้วย Temperature

การตั้งค่าการแซท (Sampling Settings)



Repeat Penalty

บางครั้งโมเดลอาจติดวนพูดคำหรือประโยคซ้ำ ๆ

- จะลดความน่าจะเป็นของคำที่เพิ่งถูกใช้ไปแล้วในบทสนทนา
- ยิ่งค่าสูง ยิ่งกันการพูดซ้ำได้มาก
- แต่ถ้าสูงเกินไปอาจทำให้ประโยคดูแปลก ๆ เพราะโมเดลพยายามหลีกเลี่ยงคำธรรมดาที่จำเป็นต้องใช้ซ้ำ

การตั้งค่าการแซท (Sampling Settings)

คำอื่น ๆ ที่ควรทราบ



Context Overflow

เกิดอะไรขึ้นเมื่อบทสนทนายาวเกิน context length



Stop Strings

ข้อความที่ถ้าโมเดลพิมพ์ออกมาแล้ว ให้หยุดตอบทันที



Structured Output

บังคับให้โมเดลตอบตามโครงสร้าง (JSON) ที่กำหนด



Prompt Template

รูปแบบการห่อ prompt ก่อนส่งเข้าโมเดล

เช่น ChatML `<|im_start|>role...<|im_end|>` ของตระกูล Qwen



Download LM Studio

Experiment with LLMs on your computer. Chat interface and programmable API.

LM Studio for Windows running x86 version 0.4.20 (build 1)



Download for Windows

0.4.20

Release notes [v0.4.20](#)

Available under the [terms of use](#)

